

National University of Computer & Emerging Sciences
Data Structures — Quiz: Array ADT and Its Implementation
Version B • Time: 15 minutes • Total marks: 10

Name: Model Solution Roll No: _____ Section: _____

Answer in the space provided. Show your working — a correct final answer with no working earns half marks.

[4 marks]

Q1. Error spotting and code output

- (a) The declaration below is copied from a slide and does not compile. Identify the error, correct it, and state which of the two dimensions may legally be omitted and why. [2]

```
Double scoreTable[][4] = { {0.5, 0.6, 0.3, 0.9},  
                             {0.6, 0.3, 0.8, 0.1} };
```

- Correction: `double scoreTable[][4] = ...;`
C++ is case-sensitive hence Double is invalid while double is correct.
- The first/row dimension can be omitted since the compiler calculates the address of `arr[i][j]` using $[base + (i \times NUM_COLS + j) \times sizeof(t)]$ where NUM-COLS is required but NUM-ROWS isn't.

- (b) This loop compiles cleanly and may still print rubbish or crash. Explain what is wrong and give the corrected loop. [2]

```
int a[5] = {1, 2, 3, 4, 5};  
for (int i = 0; i <= 5; i++)  
    cout << a[i] << " ";
```

- Problem: The loop condition is $i \leq 5$, so when $i = 5$, `a[5]` goes one element past the end of the array as valid indices are 0-4. C++ performs no bounds checking so this results in undefined behaviour i.e. it may crash, print garbage etc.
- Correction: `for (int i = 0; i < 5; i++)`
`cout << a[i] << " ";`

Q2. Address arithmetic

[3 marks]

An array is declared as `double m[3][8]`; The base address of `m` is 5000 and `sizeof(double) = 8` bytes.

- (a) How many bytes does the whole array occupy? [1]

$$\underbrace{3}_{\text{no. of elements}} \times \underbrace{8}_{\text{size of double}} \times 8 = 192 \text{ bytes (addresses ranging from 5000 to 5191)}$$

- (b) Compute the address of `m[2][5]` under row-major storage. Show the formula and the number. [1]

$$\begin{aligned} & \text{base} + (i \times \text{NUM_COLS} + j) \times \text{sizeof(double)} \\ &= 5000 + (2 \times 8 + 5) \times 8 \\ &= 5000 + (21) \times 8 \\ &= 5168 \end{aligned}$$

(c) Compute the address of the same element if the language stored the array column-major instead.

[1]

$$\begin{aligned} & \text{base} + (j \times \text{NUM-ROWS} + i) \times \text{sizeof(double)} \\ &= 5000 + (5 \times 3 + 2) \times 8 \\ &= 5000 + (17) \times 8 \\ &= 5136 \end{aligned}$$

Q3. Short answer — justify your reasoning

[3 marks]

(a) "An array is an abstract data type, and it is also a data structure." Explain what is meant at each of the two levels, using the array as your example.

[2]

- ↳ As an ADT: an ordered, fixed size, homogeneous collection whose defining operation is direct access to the element at a given position i.e. storing and retrieving by index.
- ↳ As a data structure: A contiguous block of equal-sized memory cells, with $a[i]$ computed by $[\text{base} + i \times \text{sizeof}(T)]$ and the subscript operator, allowing for $O(1)$ access.

(b) A function taking a 2-D array must be declared void f(int a[][4]) — the column count cannot be left out. Why not?

[1]

Because, the compiler uses the formula

$$\text{base} + (i \times \text{NUM-COLS} + j) \times \text{sizeof}(T)$$

for address computation of an element. NUM-COLS is required as it is the row width, so without it, the compiler cannot tell where row 'i' begins. The row count is not needed, so it may be omitted.